BULBASAUR

TEAM REFERENCE · ICPC · Edición 2026

CÓMO USAR ESTE CUADERNO

1. Copia la **plantilla base** en `main.cpp`.
2. Copia el bloque entre `COPIA` y `FIN` y pégalo antes de `main`. La plantilla ya trae las macros.
3. Lee los datos y llama a las funciones que necesites.
4. Imprime el resultado. Si el problema usa índices desde 1, suma 1 al imprimir.

COMPILAR Y EJECUTAR

```
g++ -O2 -std=c++17 prog.cpp -o prog
./prog < entrada.txt > salida.txt
```

La primera línea compila; la segunda ejecuta. Si no usas archivos, ejecuta solamente `./prog`. Las tablas indican qué hace cada función y su complejidad.

REGLAS RÁPIDAS

- Todos los índices empiezan en 0.
- Usa `long long` para contar.
- Prueba el caso mínimo antes de dudar del algoritmo.
- Revisa la lectura antes de cambiar el código.

ÍNDICE POR PROBLEMA

CAPÍTULO 1 · STRINGS p. 1
Armar y ejecutar un programa p. 1
Leer y operar strings p. 1
¿Aparece un patrón? (KMP) p. 1
Prefijos repetidos / patrón (Z) p. 1
¿Palíndromos? (Manacher) p. 2
Comparar substrings (Hash) p. 2
Distintas, ocurrencias, LCS (SAM) p. 2

CAPÍTULO 2 · JUEGOS p. 2
Ganar un juego imparcial (Grundy) p. 2
Nim y misère Nim p. 3
Dos pilas (Wythoff) p. 3
¿Gana, pierde o empata? (ciclos) p. 3

CAPÍTULO 3 · FLUJO Y MATCHING p. 3
Flujo máximo y corte mínimo (Dinic) p. 3
Matching bipartito y cover (HK) p. 3
Matching general (Blossom) p. 4

1 Strings

Un **string** es una palabra o secuencia de caracteres. En ICPC casi siempre se lee como una palabra sin espacios con `cin » s`; si la entrada trae espacios, se lee con `getline`. Los algoritmos de este capítulo responden preguntas sobre substrings, prefijos, sufijos, palíndromos y repeticiones.

1.1 Plantilla base

Todo programa empieza así. Este bloque trae las macros que usan las recetas. Pega la receta antes de `main` y cambia solo la lectura y la impresión.

```
// BLOQUE FIJO: copia esto una vez al inicio de tu programa.
#include <bits/stdc++.h>
using namespace std; using ll=long long;
#define all(x) (x).begin(),(x).end()
#define sz(x) int((x).size())
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define forn(i,a,b) for(int i=(a);i<(b);++i)
#define rforn(i,a,b) for(int i=(b)-1;i>=(a);--i)
#define INF (1LL<<60)

// ===== PEGA AQUI EL BLOQUE DE LA RECETA (ANTES DE main) =====

int main(){ios::sync_with_stdio(0);cin.tie(0);
  string s; cin>>s;              // 1) LEER los datos del problema
  int ans=sz(s);                 // 2) llamar a la funcion de la receta
  cout<<ans<<'\n';               // 3) IMPRIMIR el resultado
}
```

Ojo: Si hay varios casos, envuelve el cuerpo del `main` en `while (t-)`. No mezcles `cin » ` con `getline` sin consumir el salto de línea (ver la receta siguiente).

1.2 Operaciones de string

C++ ya trae lo básico. Las posiciones empiezan en 0.

Quiero...	Escribo...
leer una palabra	`cin » s;`
leer una línea con espacios	`getline(cin, s);`
largo	`s.size()`
carácter i	`s[i]`
copiar un pedazo	`s.substr(pos, len);`
buscar un texto	`s.find(t)`
agregar al final	`s += t;`
recorrer	`for (char c : s)`
invertir	`reverse(s.begin(), s.end());`
ordenar	`sort(s.begin(), s.end());`

Leer una línea después de un número

`cin » n` deja el salto pendiente. Consúmelo antes de `getline`:

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
string linea;
getline(cin, linea);  // ahora si lee la linea completa
```

Ojo: `s.find(t)` devuelve `string::npos` si no encuentra `t`. Si el problema cuenta desde 1, imprime `pos + 1`.

1.3 KMP: buscar un patrón

Encuentra todas las apariciones de un patrón, incluso solapadas. Entrada: `texto patron`. Salida: posiciones 0-based.

Función	Qué hace / devuelve	Tiempo
`pref(s)`	`pi[i]`: borde más largo de `s[0..i]`	$O(n)$
`kmp(text, pat)`	posiciones donde aparece `pat`	$O(n+m)$

Copiar: el bloque `COPIA KMP`. Devuelve las posiciones 0-based.

```
// ===== COPIA KMP: todo este bloque =====
vector<int> pref(const string&s){int n=sz(s);vector<int> pi(n);
  forn(i,1,n){int j=pi[i-1];
    while(j&&s[i]!=s[j])j=pi[j-1];
    if(s[i]==s[j])++j; pi[i]=j;}
  return pi;}

vector<int> kmp(const string&t,const string&p){
  if(p.empty())return {}; auto pi=pref(p); vector<int> r; int j=0;
  forn(i,0,sz(t)){while(j&&t[i]!=p[j])j=pi[j-1];
    if(t[i]==p[j])++j;
    if(j==sz(p)){r.pb(i-sz(p)+1);j=pi[j-1];}}
  return r;}
// ===== FIN KMP =====
```

El valor `pi[m-1]` es el borde más largo del patrón; el período más pequeño es `m - pi[m-1]`. Si ese período divide a `m`, el patrón es repetición de un bloque.

1.4 Z-function: prefijos repetidos

`z[i]` es el largo del prefijo más largo de `s` que coincide con `s[i..]`. Para buscar un patrón se usa `t = patron + '#' + texto`.

Función	Qué hace / devuelve	Tiempo
`zfun(s)`	arreglo `z` del string `s`	$O(n)$
`zsearch(t, p)`	posiciones de `p` en `t`	$O(n+m)$

Copiar: el bloque `COPIA Z-FUNCTION`; `zsearch` ya separa con `'#'`.

```
// ===== COPIA Z-FUNCTION: todo este bloque =====
vector<int> zfun(const string&s){int n=sz(s);vector<int> z(n);
  for(int i=1,l=0,r=0;i<n;++i){
    if(i<=r)z[i]=min(r-i+1,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]])++z[i];
    if(i+z[i]-1>r){l=i;r=i+z[i]-1;}}
  return z;}

// Busca p en t. Usa un separador que no aparezca en las strings.
vector<int> zsearch(const string&t,const string&p){
  if(p.empty())return {}; string u=p+"#"+t; auto z=zfun(u);
  vector<int> r; forn(i,sz(p)+1,sz(u))if(z[i]==sz(p))r.pb(i-sz(p)-1);
  return r;}
// ===== FIN Z-FUNCTION =====
```

El separador `'#'` debe ser un carácter que no aparezca en las strings. Si puede aparecer cualquiera, usa KMP.

1.5 Manacher: palíndromos

Todo sobre palíndromos en tiempo lineal. Entrada: una palabra sin espacios.

Función	Qué hace / devuelve	Tiempo
`Manacher(s)`	calcula los radios `d1` y `d2`	$O(n)$
`longest()`	{inicio, largo} del palíndromo más largo	$O(n)$
`count_all()`	palíndromos contando repetidos	$O(n)$

Copiar: el bloque `COPIA MANACHER`.

```
// ===== COPIA MANACHER: todo este bloque =====
struct Manacher{vector<int> d1,d2;
  Manacher(const string&s){int n=sz(s);d1.assign(n,0);d2.assign(n,0);
    for(int i=0,l=0,r=-1;i<n;++i){int k=(i>r)?1:min(d1[l+r-i],r-i+1);
      while(i-k>=0&&i+k<n&&s[i-k]==s[i+k])++k;
      d1[i]=k--; if(i+k>r){l=i-k;r=i+k;}}
    for(int i=0,l=0,r=-1;i<n;++i){
      int k=(i>r)?0:min(d2[l+r-i+1],r-i+1);
      while(i-k-1>=0&&i+k<n&&s[i-k-1]==s[i+k])++k;
      d2[i]=k--; if(i+k>r){l=i-k-1;r=i+k;}}}
  ll count_all(){ll ans=0;
    forn(i,0,sz(d1))ans+=d1[i]+d2[i]; return ans;}
  pair<int,int> longest(){int best=0,st=0;
    forn(i,0,sz(d1)){int len=2*d1[i]-1;
      if(len>best){best=len;st=i-d1[i]+1;}
      len=2*d2[i]; if(len>best){best=len;st=i-d2[i];}}
    return {st,best};}
};
// ===== FIN MANACHER =====
```

`d1[i]` cubre palíndromos impares centrados en `i`; `d2[i]` cubre los pares centrados entre `i-1` e `i`. Si piden palíndromos **distintos**, hay que eliminar duplicados.

1.6 Hash doble: comparar substrings

Compara dos pedazos en O(1) y calcula LCP en O(log n). Los rangos son `[l, r)`: incluyen `l`, no incluyen `r`.

Función	Qué hace / devuelve	Tiempo
`Hash(s)`	preprocesa s	$O(n)$
`get(l, r)`	hash de s[l..r)	$O(1)$
`equal(l1, r1, l2, r2)`	¿son iguales los dos rangos?	$O(1)$
`lcp(i, j)`	prefijo común de los sufijos i, j	$O(\log n)$

Copiar: el bloque `COPIA HASH DOBLE`.

```
// ===== COPIA HASH DOBLE: todo este bloque =====
struct Hash{
  static const ll M1=1000000007,M2=1000000009,B=911382323;
  int n; vector<ll> p1,p2,h1,h2;
  Hash(const string&s):n(sz(s)),p1(n+1,1),p2(n+1,1),h1(n+1),h2(n+1){
    forn(i,0,n){ll c=(unsigned char)s[i]+1;
      p1[i+1]=p1[i]*B%M1; p2[i+1]=p2[i]*B%M2;
      h1[i+1]=(h1[i]*B+c)%M1; h2[i+1]=(h2[i]*B+c)%M2;}}
  pair<ll,ll> get(int l,int r){
    ll a=(h1[r]-h1[l]*p1[r-l])%M1,b=(h2[r]-h2[l]*p2[r-l])%M2;
    if(a<0)a+=M1; if(b<0)b+=M2; return {a,b};}
  bool equal(int l1,int r1,int l2,int r2){
    return r1-l1==r2-l2&&get(l1,r1)==get(l2,r2);}
  int lcp(int i,int j){int lo=0,hi=min(n-i,n-j);
    while(lo<hi){int m=(lo+hi+1)/2;
      if(get(i,i+m)==get(j,j+m))lo=m;else hi=m-1;} return lo;}
};
// ===== FIN HASH DOBLE =====
```

Es probabilístico: con dos módulos el error es prácticamente nulo. Para certeza absoluta usa SAM o compara los strings.

1.7 Suffix Automaton: la estructura estrella

Resuelve casi todo sobre substrings en O(n). Usa `a`–`z`; las mayúsculas se convierten solas. Se construye **una vez**: `SAM sam(s);`.

Función	Qué hace / devuelve	Tiempo
`SAM(s)`	construye el autómata	$O(n)$
`distinct()`	substrings distintas	$O(n)$
`count(p)`	veces que aparece p	$O(\|p\|)$
`lcs(t)`	substring común más larga	$O(\|t\|)$
`kth(k)`	k-ésima substring lexicográfica	$O(26\|ans\|)$
`min_occ(k)`	substrings con al menos k apariciones	$O(n)$

Copiar: el struct completo marcado `COPIA SAM`. Aunque uses una sola función, copia todo el struct: las demás no estorban.

```
// ===== COPIA SAM: copia TODO este struct =====
struct SAM{
  static const int A=26; static const ll LIM=4000000000000000000LL;
  struct St{int len,link,next[A];
    St(int l=0,int k=-1):len(l),link(k){fill(next,next+A,-1);}};
  vector<St> st; vector<int> occ; vector<ll> ways; int last;
  static int id(char c){if(c>='A'&&c<='Z')c=c-'A'+'a';
    return(c<'a'||c>'z')?-1:c-'a';}
  SAM(const string&s=""){
    st.reserve(2*max(1,sz(s))); st.pb(St()); occ.pb(0); last=0;
    for(char c:s)extend(c);
    calc_occ(); calc_ways();}
  void extend(char ch){int c=id(ch); if(c<0)return; int cur=sz(st);
    st.pb(St(st[last].len+1)); occ.pb(1);
    int p=last;
    while(p!=-1&&st[p].next[c]<0){st[p].next[c]=cur;p=st[p].link;}
    if(p==-1)st[cur].link=0; else{int q=st[p].next[c];
      if(st[p].len+1==st[q].len)st[cur].link=q;
      else{int cl=sz(st); st.pb(st[q]);
        st[cl].len=st[p].len+1; occ.pb(0);
        while(p!=-1&&st[p].next[c]==q){st[p].next[c]=cl;p=st[p].link;}
        st[q].link=st[cur].link=cl;}}
    last=cur;}
  vector<int> order(){int n=sz(st),mx=st[last].len;
    vector<int> cnt(mx+1),o(n); forn(i,0,n)++cnt[st[i].len];
    forn(i,1,mx+1)cnt[i]+=cnt[i-1];
    rforn(i,0,n)o[--cnt[st[i].len]]=i;
    return o;}
  void calc_occ(){auto o=order(); rforn(i,1,sz(o)){
    int v=o[i]; if(st[v].link>=0)occ[st[v].link]+=occ[v];}}
  void calc_ways(){int n=sz(st); ways.assign(n,0); auto o=order();
    rforn(i,0,n){int v=o[i]; ll t=0; forn(c,0,A)if(st[v].next[c]>=0){
      t+=1+ways[st[v].next[c]]; if(t>LIM){t=LIM;break;}} ways[v]=t;}}
  ll distinct(){ll a=0;
    forn(v,1,sz(st))a+=st[v].len-st[st[v].link].len;
    return a;}
  int count(const string&p){if(p.empty())return 0; int v=0;
    for(char ch:p){int c=id(ch); if(c<0||st[v].next[c]<0)return 0;
      v=st[v].next[c];} return occ[v];}
  int lcs(const string&t){int v=0,len=0,best=0;
    for(char ch:t){int c=id(ch);
    if(c<0){v=0;len=0;continue;}
    while(v!=-1&&st[v].next[c]<0)v=st[v].link;
    if(v==-1){v=0;len=0;continue;} len=min(len,st[v].len)+1;
    v=st[v].next[c]; best=max(best,len);} return best;}
  string kth(ll k){if(k<=0||k>ways[0])return ""; string r; int v=0;
    while(1){forn(c,0,A){int u=st[v].next[c]; if(u<0)continue;
      ll cnt=1+ways[u]; if(k>cnt){k-=cnt;continue;}
      r.pb(char('a'+c)); if(--k==0)return r; v=u; break;}}}
  ll min_occ(int k){if(k<=0)return distinct(); ll a=0;
    forn(v,1,sz(st))if(occ[v]>=k)
      a+=st[v].len-st[st[v].link].len;
    return a;}
};
// ===== FIN SAM =====
```

Ojo: Memoria O($26n$). `kth` devuelve `""` si `k` no existe (vale hasta `10^18`). No agregues caracteres después de construir sin recalcular `occ`.

2 Teoría de juegos

En un juego normal, el jugador que no puede mover pierde. Las funciones de este capítulo calculan si gana el jugador que tiene el turno.

2.1 Grundy y Sprague-Grundy

Un juego imparcial se parte en componentes independientes. El SG de una posición es el `mex` de los SG de sus movimientos. Si el XOR de los SG de las componentes es distinto de 0, gana el primero.

Función	Qué hace / devuelve	Tiempo
`mex(v)`	menor entero no negativo que no está en v	$O(k)$
`subgrundy`	SG de una pila de 0 a n	$O(nm)$
`Grundy::calc()`	SG de un DAG (o {} si hay ciclo)	$O(V+E)$

Copiar: el bloque `COPIA GRUNDY`.

```
// ===== COPIA GRUNDY: todo este bloque =====
int mex(const vector<int>&v){vector<char> s(sz(v)+1);
```

```
  for(int x:v)if(x>=0&&x<=sz(v))s[x]=1;
  int g=0; while(g<sz(s)&&s[g])++g; return g;}

struct Grundy{int n; vector<vector<int>> g;
  Grundy(int n):n(n),g(n){}
  void add(int u,int v){g[u].pb(v);}
  vector<int> calc(){vector<int> in(n),o;
    forn(u,0,n)for(int v:g[u])++in[v];
    queue<int> q; forn(i,0,n)if(!in[i])q.push(i);
    while(!q.empty()){int u=q.front();q.pop();o.pb(u);
      for(int v:g[u])if(--in[v]==0)q.push(v);}
    if(sz(o)!=n)return {};
    vector<int> sg(n),mk(n+1); int t=0;
    rforn(i,0,n){int u=o[i]; ++t;
      for(int v:g[u])if(sg[v]<=n)mk[sg[v]]=t;
      int x=0; while(mk[x]==t)++x; sg[u]=x;}
    return sg;}
};

vector<int> subgrundy(int N,const vector<int>&mv){
  vector<int> sg(N+1),mk(N+1);
  forn(x,1,N+1){for(int m:mv)if(m<=x)mk[sg[x-m]]=x;
    int g=0; while(mk[g]==x)++g; sg[x]=g;}
  return sg;}
// ===== FIN GRUNDY =====
```

Ojo: Para varias pilas o componentes, combina con XOR: `g = g1 ^ g2 ^ ...`. Si `g != 0` gana el primero. Un estado terminal tiene SG 0.

2.2 Nim y misère Nim

Varias pilas; en cada turno se quita cualquier cantidad positiva de una sola pila. En misère, el que toma la última piedra pierde.

Función	Qué hace / devuelve	Tiempo
`nim_win(piles)`	¿gana el primero?	$O(n)$
`misere(piles)`	versión misère	$O(n)$
`nim_move(piles)`	{pila, nuevo tamaño} que deja XOR 0	$O(n)$

Copiar: el bloque `COPIA NIM`.

```
// ===== COPIA NIM: todo este bloque =====
bool nim_win(const vector<int>&p){int x=0;
  for(int a:p)x^=a; return x;}

bool misere(const vector<int>&p){int x=0; bool one=1;
  for(int a:p){x^=a; if(a>1)one=0;}
  return one?x==0:x;}

// {indice, nuevo tamano} para dejar XOR = 0; {-1,-1} si no hay.
pair<int,int> nim_move(const vector<int>&p){int x=0; for(int a:p)x^=a;
  if(!x)return {-1,-1}; forn(i,0,sz(p)){int t=p[i]^x;
    if(t<p[i])return {i,t};} return {-1,-1};}
// ===== FIN NIM =====
```

Ojo: En misère, si todas las pilas son 0 o 1, gana el primero cuando la cantidad de pilas es par. En otro caso se juega como Nim normal.

2.3 Wythoff: dos pilas

Se puede quitar cualquier cantidad de una pila, o la misma cantidad de las dos. Las posiciones perdedoras son $(\lfloor k\varphi \rfloor, \lfloor k\varphi \rfloor + k)$.

Función	Qué hace / devuelve	Tiempo
`floor_phi(d)`	$\lfloor d\varphi \rfloor$ sin error de coma flotante	$O(\log d)$
`wythoff(a, b)`	¿gana el primero?	$O(\log d)$

Copiar: el bloque `COPIA WYTHOFF`.

```
// ===== COPIA WYTHOFF: todo este bloque =====
ll floor_phi(ll d){ll lo=0,hi=2*d+1;
  while(lo+1<hi){ll m=(lo+hi)/2;
    __int128 a=(__int128)2*m-d;
    bool le=((__int128)2*m<=d)||(a*a<=(__int128)5*d*d);
    if(le)lo=m; else hi=m;}
  return lo;}

bool wythoff(ll a,ll b){if(a>b)swap(a,b); return a!=floor_phi(b-a);}
// ===== FIN WYTHOFF =====
```

Ojo: El código usa `__int128` y búsqueda binaria para no depender de la precisión de `double`. Si `a == floor\_phi(b-a)` (con `a <= b`), la posición es perdedora.

2.4 Juegos con ciclos: análisis retrógrado

Un estado sin movimientos pierde. Si puedo mover a un estado perdedor, gano; si todos mis movimientos llevan a estados ganadores, pierdo. Lo que queda es empate.

Función	Qué hace / devuelve	Tiempo
`solve_game(n, g)`	1 = gana, -1 = pierde, 0 = empate	$O(V+E)$

Copiar: el bloque `COPIA JUEGOS CON CICLOS`.

```
// ===== COPIA JUEGOS CON CICLOS: todo este bloque =====
// state: 1 = gana, -1 = pierde, 0 = empate.
vector<int> solve_game(int n,const vector<vector<int>>&g){
  vector<int> out(n),st(n); vector<vector<int>> rev(n);
  forn(u,0,n)for(int v:g[u]){++out[u]; rev[v].pb(u);}
  queue<int> q; forn(v,0,n)if(!out[v]){st[v]=-1;q.push(v);}
  while(!q.empty()){int v=q.front();q.pop();
    for(int u:rev[v]){if(st[u])continue;
      if(st[v]==-1){st[u]=1;q.push(u);}
      else if(--out[u]==0){st[u]=-1;q.push(u);}}}
  return st;}
// ===== FIN JUEGOS CON CICLOS =====
```

Ojo: Sirve para juegos con tableros y movimientos que pueden repetir estados. Si el grafo es un DAG, no hay empates y puedes usar Grundy en vez de esta función.

3 Flujo y matching

Dinic resuelve flujo con capacidades. Para matching bipartito usa Hopcroft-Karp; para grafos generales, Blossom.

3.1 Dinic: flujo máximo y corte mínimo

Grafo dirigido con capacidades `long long`. Devuelve el flujo máximo. Después del flujo, `cut(s)` marca el lado S del corte mínimo.

Función	Qué hace / devuelve	Tiempo
`Dinic(n)`	crea un grafo con `n` nodos	$O(n)$
`add(u, v, cap)`	agrega arista dirigida con capacidad	$O(1)$
`maxflow(s, t)`	valor del flujo máximo	$O(V^2E)$
`cut(s)`	lado S después de saturar	$O(V+E)$
`cut_edges(s)`	aristas originales que cruzan el corte	$O(V+E)$

Copiar: el bloque `COPIA DINIC`. Usa `cut` y `cut_edges` después de `maxflow`.

```
// ===== COPIA DINIC: todo este bloque =====
struct Dinic{
  struct E{int to,rev; ll cap;}; struct R{int u,v; ll cap;};
  int n; vector<vector<E>> g; vector<R> raw; vector<int> lv,it;
  Dinic(int n):n(n),g(n),lv(n),it(n){}
  void add(int u,int v,ll c){raw.pb({u,v,c});
    E a{v,sz(g[v]),c},b{u,sz(g[u]),0}; g[u].pb(a); g[v].pb(b);}
  bool bfs(int s,int t){fill(all(lv),-1); queue<int> q;
    lv[s]=0; q.push(s);
    while(!q.empty()){int v=q.front();q.pop();
      for(auto&e:g[v])if(e.cap>0&&lv[e.to]<0){
        lv[e.to]=lv[v]+1; q.push(e.to);}}
    return lv[t]>=0;}
  ll dfs(int v,int t,ll f){if(v==t)return f;
    for(int&i=it[v];i<sz(g[v]);++i){auto&e=g[v][i];
      if(!e.cap||lv[e.to]!=lv[v]+1)continue;
      ll x=dfs(e.to,t,min(f,e.cap)); if(!x)continue;
      e.cap-=x; g[e.to][e.rev].cap+=x; return x;}
    return 0;}
  ll maxflow(int s,int t){ll f=0;
    while(bfs(s,t)){fill(all(it),0); while(ll x=dfs(s,t,INF))f+=x;}
    return f;}
  vector<char> cut(int s){vector<char> vis(n); queue<int> q;
    vis[s]=1; q.push(s);
    while(!q.empty()){int v=q.front();q.pop();
      for(auto&e:g[v])if(e.cap>0&&!vis[e.to]){
        vis[e.to]=1; q.push(e.to);}}
    return vis;}
  vector<pair<int,int>> cut_edges(int s){auto vis=cut(s);
    vector<pair<int,int>> r;
    for(auto&e:raw)if(vis[e.u]&&!vis[e.v])r.pb({e.u,e.v});
    return r;}
};
// ===== FIN DINIC =====
```

Ojo: Para capacidades unitarias en una red bipartita, Hopcroft-Karp es más simple. Para vértices con capacidad, divide cada vértice en entrada y salida con una arista de capacidad.

3.2 Hopcroft-Karp: matching bipartito

Matching máximo en un grafo bipartito. Izquierda: `0..n-1`; derecha: `0..m-1`. El cover usa `0..n-1` para la izquierda y `n..n+m-1` para la derecha.

Función	Qué hace / devuelve	Tiempo
`HK(n, m)`	crea las dos particiones	$O(n+m)$
`add(u, v)`	arista izquierda u – derecha v	$O(1)$
`matching()`	tamaño del matching máximo	$O(E\sqrt{V})$
`cover()`	cover mínimo (Kőnig)	$O(V+E)$
`indep()`	conjunto independiente máximo	$O(V+E)$
`path_cover`	cobertura mínima de caminos en un DAG	$O(E\sqrt{V})$

Copiar: el bloque `COPIA HOPCROFT-KARP`.

```
// ===== COPIA HOPCROFT-KARP: todo este bloque =====
struct HK{
  int n,m; vector<vector<int>> g; vector<int> pu,pv,d;
  HK(int n,int m):n(n),m(m),g(n),pu(n,-1),pv(m,-1),d(n){}
  void add(int u,int v){g[u].pb(v);}
  bool bfs(){queue<int> q;
    forn(u,0,n){if(pu[u]<0){d[u]=0;q.push(u);}else d[u]=-1;}
    bool f=0; while(!q.empty()){int u=q.front();q.pop();
      for(int v:g[u]){int w=pv[v];
        if(w<0)f=1; else if(d[w]<0){d[w]=d[u]+1;q.push(w);}}}
    return f;}
  bool dfs(int u){for(int v:g[u]){int w=pv[v];
      if(w<0||(d[w]==d[u]+1&&dfs(w))){pu[u]=v;pv[v]=u;return 1;}}
    d[u]=-1; return 0;}
  int matching(){int r=0;
    while(bfs())forn(u,0,n)if(pu[u]<0&&dfs(u))++r; return r;}
  vector<int> cover(){vector<char> a(n),b(m); queue<int> q;
    forn(u,0,n)if(pu[u]<0){a[u]=1;q.push(u);}
    while(!q.empty()){int u=q.front();q.pop();
      for(int v:g[u]){if(pu[u]==v||b[v])continue; b[v]=1;
        if(pv[v]>=0&&!a[pv[v]]){a[pv[v]]=1;q.push(pv[v]);}}}
    vector<int> r; forn(u,0,n)if(!a[u])r.pb(u);
    forn(v,0,m)if(b[v])r.pb(n+v); return r;}
  vector<int> indep(){auto c=cover(); vector<char> u(n+m);
    for(int x:c)u[x]=1; vector<int> r;
    forn(i,0,n+m)if(!u[i])r.pb(i); return r;}
};
int path_cover(int n,const vector<pair<int,int>>&e){
  HK h(n,n); for(auto&p:e)h.add(p.fi,p.se); return n-h.matching();}
// ===== FIN HOPCROFT-KARP =====
```

Ojo: Si solo necesitas el tamaño del matching, `matching()` es suficiente. El cover y el independiente son complementarios: su tamaño suma `n + m`.

3.3 Blossom: matching general

Matching máximo en un grafo no bipartito (Edmonds). `add` agrega la arista en ambos sentidos. Para `n` hasta unos 500.

Función	Qué hace / devuelve	Tiempo
`Blossom(n)`	crea el grafo	$O(n)$
`add(u, v)`	arista no dirigida	$O(1)$
`matching()`	tamaño del matching máximo	$O(V^3)$
`mt[i]`	pareja de i, o -1	$O(1)$

Copiar: el bloque `COPIA BLOSSOM`.

```
// ===== COPIA BLOSSOM: todo este bloque =====
struct Blossom{
  int n; vector<vector<int>> g; vector<int> mt,p,b,q;
  vector<char> used,bl;
  Blossom(int n):n(n),g(n),mt(n,-1),p(n),b(n),used(n),bl(n){}
  void add(int u,int v){g[u].pb(v); g[v].pb(u);}
  int lca(int a,int c){vector<char> s(n);
    while(1){a=b[a];s[a]=1;if(mt[a]<0)break;a=p[mt[a]];}
    while(1){c=b[c];if(s[c])return c;c=p[mt[c]];}}
  void mark(int v,int x,int ch){while(b[v]!=x){
      bl[b[v]]=bl[b[mt[v]]]=1; p[v]=ch; ch=mt[v]; v=p[mt[v]];}}
  int find(int root){used.assign(n,0); p.assign(n,-1);
    forn(i,0,n)b[i]=i; q.clear(); q.pb(root); used[root]=1;
    forn(qi,0,sz(q)){int v=q[qi];
      for(int to:g[v]){if(b[v]==b[to]||mt[v]==to)continue;
        if(to==root||(mt[to]>=0&&p[mt[to]]>=0)){
          int x=lca(v,to); bl.assign(n,0); mark(v,x,to); mark(to,x,v);
          forn(i,0,n)if(bl[b[i]]){b[i]=x;
            if(!used[i]){used[i]=1;q.pb(i);}}}
        else if(p[to]<0){p[to]=v; if(mt[to]<0)return to;
          to=mt[to]; used[to]=1; q.pb(to);}}}
    return -1;}
  int matching(){mt.assign(n,-1);
    forn(i,0,n){if(mt[i]>=0)continue; int v=find(i);
      while(v>=0){int x=p[v],y=mt[x]; mt[v]=x; mt[x]=v; v=y;}}
    int r=0; for(int x:mt)if(x>=0)++r; return r/2;}
};
// ===== FIN BLOSSOM =====
```

Ojo: En un grafo bipartito no uses Blossom: Hopcroft-Karp es mucho más rápido. Blossom sirve para grafos generales con ciclos impares.